CSCI 532 - Part 3: Implementation and Evaluation of Enhanced Federated Learning with PID

Kody Rogers
Cody Smith
Nicholas Gouldin
Rochester Institute of Technology

1 Introduction

This report details the implementation and evaluation of enhanced federated learning with a Proportional-Integral-Derivative (PID) anomaly detection mechanism. Building upon the data collected in Part 1 and Part 2 this phase implementation serves as a baseline for comparison with the PID enhanced FL system. The objective of the PID system is to increase the validity against the malicious clients by dynamically excluding clients with anomalies to their models.

2 Experimental Setup

2.1 Framework and Tools

Our implementation utilizes the following key components:

- **Framework:** Flower for federated learning orchestration
- **Deep Learning:** PyTorch for model development and training
- **Data Processing:** NumPy for numerical operations
- **Visualization:** Matplotlib for plotting metrics

We chose Flower as our FL framework due to its robust support for customization, particularly in metric logging and aggregation strategy implementation. The framework's Python-native design integrates seamlessly with PyTorch and our existing data processing pipeline.

2.2 System Architecture

Our implementation follows a centralized FL architecture with an integrated PID-based anomaly detection mechanism. The system consists of the following components:

- **Central Server:** Coordinates training, computes the centroid of all client models, calculates PID scores, and excludes clients exceeding the threshold before aggregation.
- **Clients:** Independently train local models using private datasets and send updated weights to the server each round.
- **PID Controller Module:** Computes the PID metrics ($D(t)$, $\sum D(\tau)$, and $D(t)-D(t-1)$) and determines if a client should be included in aggregation.

Although executed in a single-machine simulation, this setup accurately represents distributed FL communication and anomaly-based filtering.

2.3 PID-Based Enhancement

The PID controller was integrated into the aggregation process on the server side. At each communication round, the server calculates the Euclidean distance $D(t)$ between each client's model weights and the global centroid. The PID score is computed as:

$$PID(t) = K_p D(t) + K_i \sum_{\tau=1}^{t} D(\tau) + K_d(D(t) - D(t-1))$$

in this function the K_p, K_i, and K_d are the proportional, integral, and derivative coefficients respectively. We began experimentation with $(K_p, K_i, K_d) = (1, 0.1, 0.5)$.

Clients whose PID score exceeds a threshold are excluded from aggregation in the subsequent round. This allows the FL server to adaptively filter out malicious or inconsistent model updates over time.

2.4 Dataset Description

We utilize the MedMNIST dataset prepared in Part 1, with the following characteristics:

- Distribution across 20 clients
- Balanced allocation of samples per class
- Standardized preprocessing including normalization and augmentation

2.5 Model Architecture

Our implementation uses a Convolutional Neural Network (CNN) designed for medical image classification. The architecture is detailed in Table 1.

Table 1: Model Architecture

Layer	Configuration	Output Shape
Conv2D	3×3, 32 filters	28×28×32
BatchNorm	-	28×28×32
ReLU	-	28×28×32
MaxPool2D	2×2	14×14×32
Conv2D	3×3, 64 filters	14×14×64
BatchNorm	-	14×14×64
ReLU	-	14×14×64
MaxPool2D	2×2	7×7×64
Flatten	-	3136
Dense	128 units	128
ReLU	-	128
Dense	num_classes	num_classes

2.6 Implementation Details

The implementation includes the following key configurations:

- **Global Model:** Initialized with Kaiming initialization

- **Local Training:** 5 epochs per round, batch size 32
- **Aggregation:** FedAvg algorithm
- **Communication:** 100 rounds total
- **Metrics:** Per-client and averaged loss/accuracy tracked each round

3 Evaluation Without Attack

3.1 Procedure

We first evaluate the FL system under normal conditions, without any malicious clients or data poisoning. The evaluation consists of:

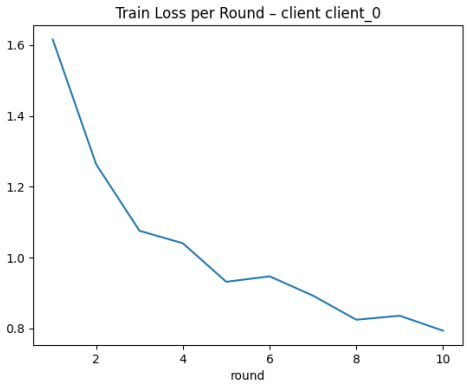
- 10 communication rounds
- run the simulation using
- `py .\launcher.py --epochs 3`
- Continuous logging of loss and accuracy metrics
- Both per-client and global model performance tracking

3.2 Results (Plots & Metrics)

This data was generated using the following command

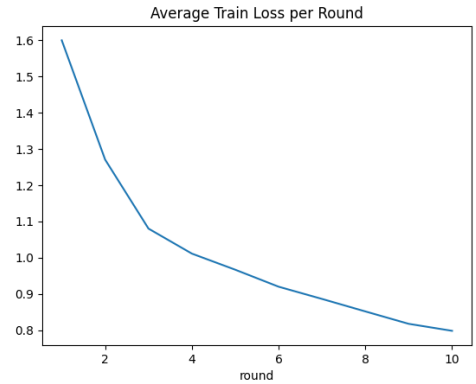
`python launcher.py --epochs 3`

- Average Training Loss per client vs rounds
  - Client 0 started with a 1.6 training loss in round 1
  - Client 0 had roughly a 0.9 training loss by round 5
  - And by round 10 Client 0 had a 0.78 training loss
  - Image



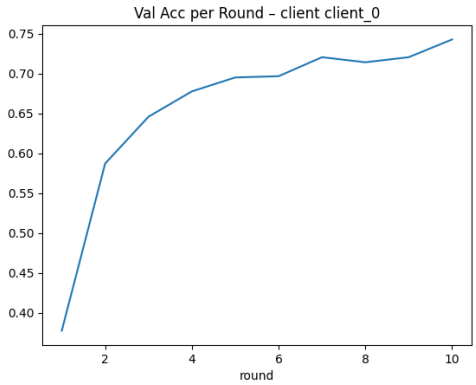
  –

- Average loss vs rounds
  - Client average started with a 1.67 training loss in round 1
  - The client average had a 0.88 training loss by round 5
  - And by round the client average had a 0.78 training loss
  - Image



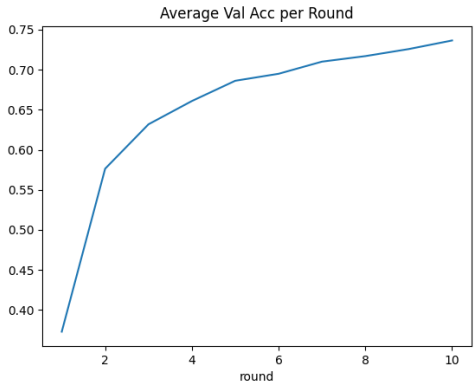
  –

- Accuracy per client vs rounds
  - Client 0 has an accuracy of 37% in round 1
  - Client 0 had an accuracy of 69% by round 5
  - And after round 10 Client 0 had an accuracy of 74%
  - Image



  –

- Average accuracy vs rounds
  - The average client had an accuracy of 37% in round 1
  - The average client had an accuracy of 68% by round 5
  - And after round 10 Client 0 had an accuracy of 73%
  - Image



  –

3.3 Analysis

The results show that all clients improved over time. Training loss kept going down and leveled out around 0.78, while accuracy went up from about 37% to over 70%. This means the model was learning well and the clients were working together effectively. The results also look pretty consistent between clients, showing the training stayed balanced.

4 Evaluation With Attack

4.1 Attack Description

We implement a data poisoning attack through label flipping with the following parameters:

- Attack type: Label flipping
- Number of poisoned clients: [2]
- Label flipping percentage: [50%]

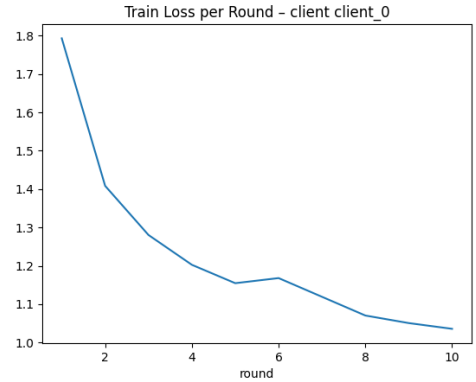
4.2 Procedure

The attack evaluation follows these steps:

- Selection and poisoning of target clients
  - We ran the command
  - `py .\poison_clients.py --clients .\client_0.pt .\client_1.pt --flip-rate 0.5`
  - this flipped labels 1 and 8 in the training data by a set rate of 50%
- Running FL simulation with poisoned clients
  - run the simulation using this command
  - `py .\launcher.py --attack --epochs 3`
- Collection of comprehensive metrics

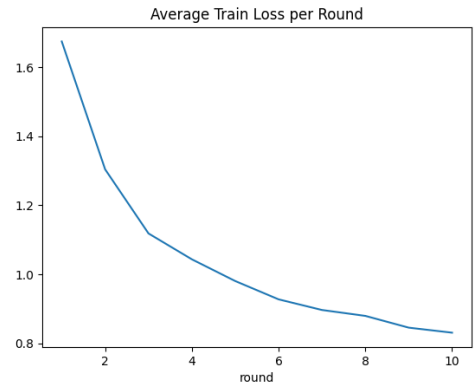
4.3 Results (Plots & Metrics)

- Training value loss per client vs rounds (poisoned scenario Client 0)
  - In this case Client 0 was poisoned
  - We saw Client 0 have a training loss of 1.86 which is already much higher than the training loss in the non poisoned set of 1.6 and well above the average 1.74 for the first round
  - Much of the same continues for the entirety of the program by round 5, Client 0 has 1.15 training loss compared to the average of 0.93
  - By round 10 the average was 0.8 and the poisoned Client 0 had 1.03 training value loss



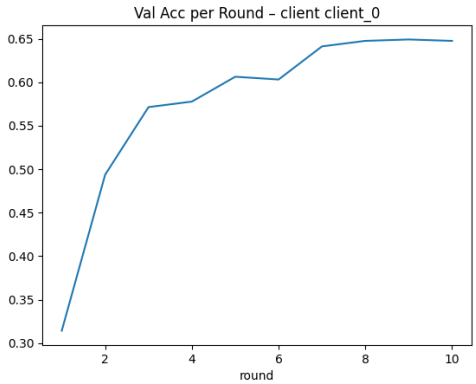
  -

- Average loss vs rounds
  - The average loss was similar to the non poisoned data just running slightly higher due to the two outliers
  - Round 1 had a 1.74 Training loss, Round 5 0.93 training loss, and Round 10 0.8 Training loss

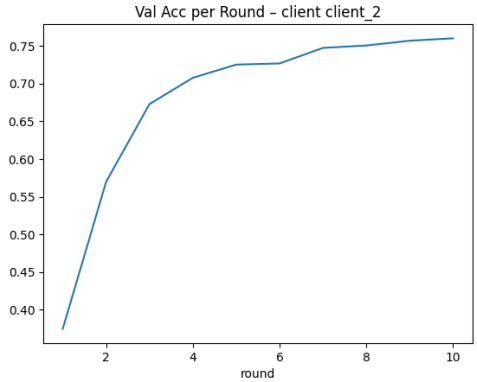


  -

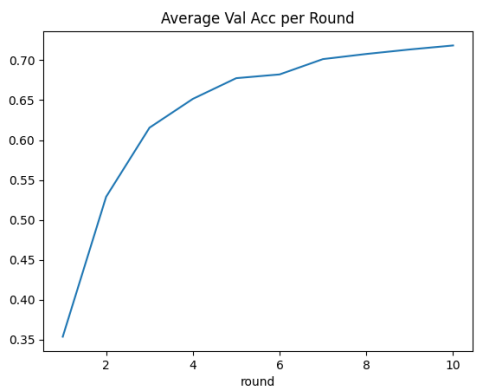
- Accuracy per client vs rounds
  - Client 0 accuracy also was much below the average as the accuracy was reduced to the label swapping attack it couldn't properly train
  - Here is a comparision from Client 0 and Client 2 where Client 0 was poisoned and Client 2 was not



  -



–
  - you can see the difference in the Accuracy per round as Client 0 did not even reach 65% accuracy while Client 2 was near the Average Accuracy of 75%
- Average accuracy vs rounds
  - Similarly to the Average Training loss the Average accuracy slightly off in this case lower



–
  - The overall accuracy is slightly below the non attack accuracy which is to be expected from only two clients.

4.4 Analysis

The poisoned Client 0 had a higher training loss and lower accuracy than the other clients. The attack mostly affected Client 0, while the overall average loss and accuracy only dropped slightly. Which is to be expected with only 2 clients being poisoned (Client 0 and Client 1). If more clients were poisoned we can expect to see the data averages go up for the training loss and down for the accuracy as the AI will struggle to determine the correct data.

5 PID Evaluation

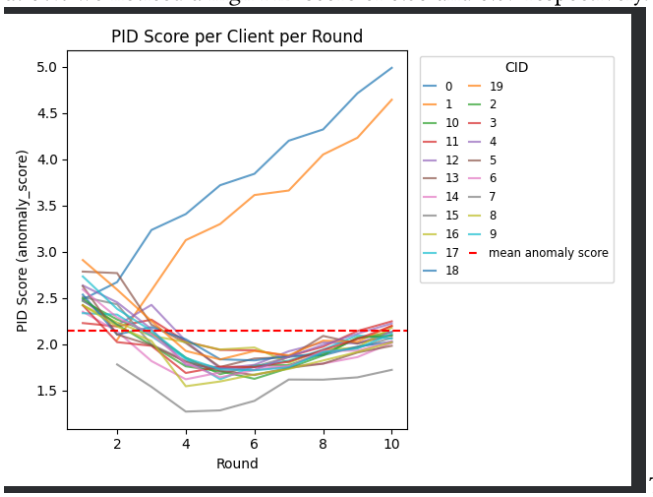
In this section we evaluated the PID based anomaly detection we implemented into our Federated Learning Framework. The goal of this evaluation is to see how well our PID system can detect poisoned clients. For our PID configuration we used the following coeffients after our preliminary tuning

- Kp = 1.0
- Ki = 0.1
- Kd = 0.05

Our pid threshold defaults to 2.79, which seems to produce the most consistent anomaly detection results

5.1 PID Score per Client per Round

When we analyzed the attack using our group's set values for the poisoned data with client 0 and 1 having their data flipped at 50% we noticed a high PID score of 5.35 and 5.39 respectively.

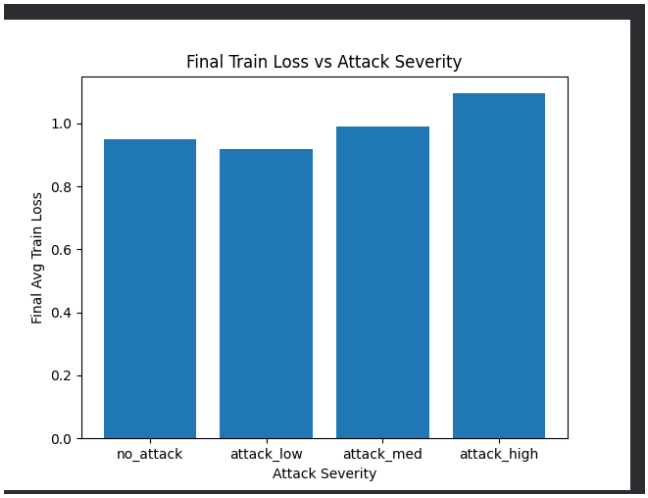


The result shows a clear separation between the normal clients and the poisoned clients which had an average PID score of about 2.2 which is under our PID threshold. This shows that the PID mechanism was implemented successfully able to detect the poisoned clients and deviate them from the normal unmodified clients.

5.2 Final Train Loss vs Attack Severity

Next is the Final Training Loss vs Attack Severity, we separated our data into 4 different categories see section 6.2 for further setup. The 4 categories are, No Attack, Low Attack, Medium Attack, and High Attack. Each of the data has different amounts of clients poisoned and different flip rates.

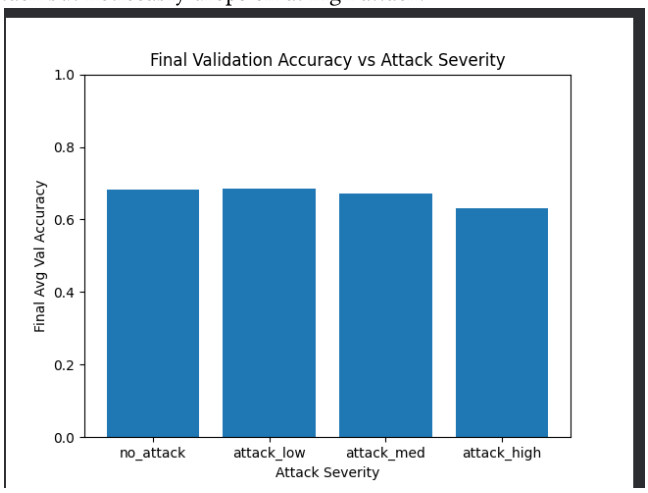
- No Attack - No Poisoned Clients, No Flip Rate
- Low Attack - 3 out of 20 Poison Clients, 25% Flip Rate
- Medium Attack - 5 out of 20 Poison Clients, 35% Flip Rate
- High Attack - 8 out of 20 Poison Clients, 60% Flip Rate



This graphic shows how stronger data poisoning causes the model to struggle more during the training and from a PID perspective the system is able to detect that the data is becoming less and less stable as the attack intensity grows.

5.3 Final Validation Accuracy vs Attack Severity

Finally the Accuracy vs the attack severity, like the last section we kept our data separated into the 4 categories. The validation of the accuracy remains relatively stable from No Attack to Medium Attack but noticeably drops off at high attack.



From a PID stand point the pattern is consistent with what we've seen above where more data is being loss training due to the high data poisoning. When the poisoning is small the model is able to detect anomalies better in comparison to the larger attacks that hurt data generalization.

6 Implementation and Reproducibility

6.1 Code Structure

The implementation is organized as follows:

```
src/
   clients_backup/
      client_0.pt
      ...
clients_data/
   client_0_images/
       sample_0_label0.png
       sample_0_label1.png
       ...
       sample_0_label8.png
   client_1_images/...
   client_0.pt
   ...
   client_19.pt
results/
   attack/
       attack_avg_train_loss.png
       attack_avg_val_acc.png
       attack_train_loss_client_0.png
       ...
       attack_train_loss_client_19.png
       attack_val_acc_client_0.png
       ...
       attack_val_acc_client_19.png
       per_client_eval.csv
       per_client_train.csv
       round_averages.csv
   attack_high/
       Same format as above.csv
   attack_low/
   attack_med/
   no_attack/
       no_attack_avg_train_loss.png
       no_attack_avg_val_acc.png
       no_attack_train_loss_client_0.png
       ...
       no_attack_train_loss_client_19.png
       no_attack_val_acc_client_0.png
       ...
       no_attack_val_acc_client_19.png
       per_client_eval.csv
       per_client_train.csv
       round_averages.csv
   summary/
       accuracy_vs_attack_severity.png
       loss_vs_attack_severity.png
       setup.txt
src/
    clean_data.py
    clients_fl.py
    convert_clients.py
    dataset_model.py
    launcher.py
    plot_results.py
    plot_severity_to_performance.py
    poison_clients.py
    restore_clients.py
    server_fl.py
    utils_logging.py
```

6.2 Setup Instructions

To reproduce our results:

(1) **Install dependencies:**

```
pip install -r requirements.txt
```

(2) **Clean the dataset:**

```
python clean_data.py
```

This step downloads the data (if not provided), normalizes it, and removes duplicates and blank samples.

(3) **Convert data to client format:**

```
python convert_clients.py
```

This partitions the data into client-specific datasets inside the `client_data` folder.

(4) **Run the federated learning launcher:**

```
python launcher.py [-h] [--attack] [--rounds
ROUNDS] [--epochs EPOCHS] [--clients CLIENTS]
```

- Note: Do NOT use `–poisoned_clients` or `–flip_frac` alone; use `poison_clients.py` to apply attacks.

(5) **Poison client data (optional):**

```
python poison_clients.py --clients client_0.pt
client_1.pt --flip-rate 0.5
```

This flips the labels for clients 1 and 8 at the specified fraction. To revert changes:

```
python restore_poison.py
```

(6) **Visualize results:**

```
python plot_results.py --mode {no_attack, attack}
```

(7) **Plotting Attack Severity vs Loss/Accuracy**
- Run 4 separate simulations of the previous steps, each simulation running poison_clients.py at a specific –flip-rate value:
- Please run AT LEAST 20 rounds to ensure that the pid system has time to detect the malicious clients
- It is assumed that you delete data in clients_data and use convert_clients.py to produce fresh data to poison.

(8) **Levels of flipped labels for each attack type:**
- no_attack 0.00% (baseline)
- attack_low
  - flip-rate 0.25
  - 3 clients out of 20 poisoned (clients 0-2)
- attack_med
  - flip-rate 0.35
  - 5 clients out of 20 poisoned (clients 0-4)
- attack_high
  - flip-rate 0.6
  - 8 clients out of 20 poisoned (clients 0-7)

(9) **Finally: Run plot_severity_to_performance.py**
- A chart giving the differences in accuracy and loss for each attack type will be saved

7 Conclusion

Federated Learning is generally a great way to train data on a specific dataset. In all of our unpoisoned tests, the validation accuracy exceeded 70%, and the training loss was well below 1.0. However, if the data is poisoned or corrupted with flipped labels, it can lose a significant amount of accuracy. In the poisoned tests, the loss lowered to the a similar amount, but far less consistently. The accuracy also dipped to between 60-70%. This shows that while Federated Learning is good when training on isolated datasets, but is not exceptionally resilient when working with poisoned data. These limitations of the Federated Learning structure are something that the PID-enhanced FL system has attempted to fix but still seeing issues with high amounts of data corruption. The PID system is good for detecting these issues and phasing some out of the poisoned data but with high amount of poisoning show issues.

8 Teamwork and Effort

Table 2: Team Member Contributions

Member	Tasks	Hours
Kody Rogers	[Data Analyst, Writeup]	[8 Hours]
Cody Smith	[Coding, Debugging]	[10 hours]
Nicholas Gouldin	[Writeup]	[5 Hours]